Part 1: Program and Core Logic Overview

I managed to create a 3-day weather app using live weather data from Open-Meteo, a free weather internet service using Calgary's exact coordinates. If successful, triggerWeatherSyncFetch grabs the data, saves it to the app and then shows it to the screen. If it catches error, it hides the weather card and displays "Data offline" error message on screen. However, if the app is still waiting for the asynchronous application to complete, the screen displays "Loading Location..."

Part 2: Explain Up to Three Functions

1. interpretWmoCode(code)

Purpose: A data translation component designed to convert standardized, numeric international weather condition codes into plain-language English descriptors.

Input: A single numeric argument (code) representing an observed weather state (e.g., 0, 3, 61).

Process: An internal conditional matching matrix (switch) that maps individual numbers or grouped ranges of numbers into defined meteorological classifications.

Output: A standardized text string containing a descriptive weather name (e.g., "Sunny", "Cloudy", "Rain", or "Thunderstorm").

Connections: Links directly to the data stream provided by the Open-Meteo payload structure, feeding its string outputs directly into executeDOMWidgetRender for structural assembly.

Failure behavior: Features an unconditional fallback case (default) that automatically assigns and returns the string value "Cloudy" whenever an unmapped or invalid integer is supplied.

2. evaluateScaleMetric(celsiusRawVal, appendUnitSymbol)

Purpose: A mathematical utility blueprint structured to trim decimal readings and handle conditional thermal unit system modifications.

Input: A primary numeric or decimal value (celsiusRawVal), an optional configuration flag (appendUnitSymbol), and read-only access to the external state toggle appState.isCelsiusSystem.

Process: A numerical parser that truncates float values into base-10 integers, evaluated against an active system state boolean to conditionally run a standard Fahrenheit conversion formula: (Value × 9/5) + 32.

Output: A text value appending a degree symbol to a rounded temperature integer (e.g., "22°"), or a raw unformatted number depending on the configuration flags.

Connections: Intersects directly with the configuration profile appState and is actively embedded inside executeDOMWidgetRender to populate display elements.

Failure behavior: Offers no built-in data type filtering; corrupted inputs or non-numeric strings cause the internal mathematical engine to output NaN ("Not a Number") directly to the screen elements.

3. triggerWeatherSyncFetch(locationTarget)

Purpose: An asynchronous network pipeline module responsible for querying live environmental data from a remote endpoint and updating application state storage.

Input: An optional string variable (locationTarget), which is superseded at runtime by localized, immutable coordinate values.

Process: An asynchronous operation that suppresses visible warning elements, initiates a secure HTTP handshake with the Open-Meteo API using hardcoded regional coordinates (51.0447, -114.0719), maps the incoming stream to a JSON object, and mutates appState.rawPayload.

Output: Modifies the global appState memory stack and alters the visual parameters of the webpage layout by instigating downstream layout paints.

Connections: Strongly dependent on external Open-Meteo cloud infrastructure, the document element structures error-message and location-heading, and the initialization sequence executeDOMWidgetRender.

Failure behavior: Bound inside an active exception-handling ecosystem (try/catch) that catches network disruptions or server offline statuses, suppresses display assets, unhides the layout container error-message, and forces the location header to print "Data Offline".

Part 3: Code Decisions and Optimization

1. Why did you organize the code into its current functions or sections?
   While there’s no single standard or perfect way to write code, I believe that this code is organized effectively because it splits everything into clear, common-sense sections: a tiny notepad at the top to remember my choices (appState), a visual dictionary for the art design, pure math and translator functions that don't mess with the screen, and a dedicated setup for painting the webpage. By keeping the unpredictable internet requests isolated from the rest of the code, a web crash won't break the whole app, and using a single notebook means all my temperatures change instantly without getting out of sync. It is a perfect, lightweight setup for a

simple one-page project, though I'd eventually have to slice it up into multiple files if I wanted to scale it up into a massive, multi-city app with a massive team of developers.

2. What did you change to improve performance or avoid unnecessary work?
   I changed the sources/links that aren't working for me. Specifically, the //iconify.design and the //wttr.in. I changed them into my current sources and they worked just fine.

3. What alternative approach did you consider, and why did you choose your final approach?
   I changed the overall look of my app. AI gave a standard design for a widget but I researched how I wanted mine to look and designed it how I actually want it.

Part 4: One Challenge and What You Learned

I was so confused why my App was still offline eventho I already changed my weather data source. My code is "const apiEndpointUrl = `https://api.open-meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current=temperature_2m,weather_code&daily=weather_code,temperature_2m_max,temperature_2m_min&timezone=America%2FEdmonton`;"

I found out that my link is broken because I forgot to put $ after "=". I realized that small details like this can really be a big issue especially when you don't know what you're looking for and when you're running out of time. Every syntax is important and AI help can be a big help in saving time and energy.

Part 5: AI Use and Evaluation

1. Which parts of the assignment did you use AI to support?
   I asked AI to explain the main logic in creating a weather app cause I could just copy some random developer from youtube how to create a weather app and not understand it. AI helped me how to create functions to fetch data (triggerWeatherSyncFetch), translate the number data it gives me into words (interpretWmoCode), and gives visuals to my app depending on what data is given to me (getThemeKey, assignWeatherSymbol, applyThematicInterfaceStyles).

2. How did you determine whether an AI suggestion was correct?
   I let it explain what it's function and double check it in MDN. I also test it everytime if it works on my page before settling down.

3. What AI suggestion did you change, reject, or question, and why?
   Its standard design template. I never settle on whatever design AI is giving me cause they are very common and everyone in our class who didn't tweak their codes will have a weather app

that looks the same. I make sure that eventho I asked AI's help for functions, the app still doesn't feel generic.

Part 6: One Future Programming Takeaway

I think it's okay to use AI for assistance. It's a great help in being efficient in handling time and energy but at the end of the day, it should only be a help, not the main creator of the codes. A developer should get the logic of the project and go from there.